



Final Project Presentation

Asset-backed tokenization, vaultyield, AMM liquidity, and on-chain governance

Students: Yergalyuly Daniyar, Talasbek Kasymbek, Akhmetov Nurali
Group: SE-2404

- Tech stack: React · Foundry · Solidity · The Graph (Subgraph)
- Local Anvil demo & L2-ready deployment path
- Final blockchain course project — full-stack, testable, auditable

Problem

Real-world asset (RWA) protocols require canonical tokenization, liquid markets, transparent governance, and auditable accounting to be credible and compliant.

Solution



Tokenized asset representation

On-chain tokens map to off-chain assets with verifiable metadata and AssetBadgeNFT proofs.



Vault-based deposits & shares

ERC4626-style accounting issues shares, accrues yield, and preserves audit trails.



AMM liquidity routing

Constant-product pools enable swaps and on-chain price discovery with slippage and fee controls.



Governance-controlled changes

Proposal → timelock → execute workflow ensures deliberate upgrades and treasury actions.

System Architecture

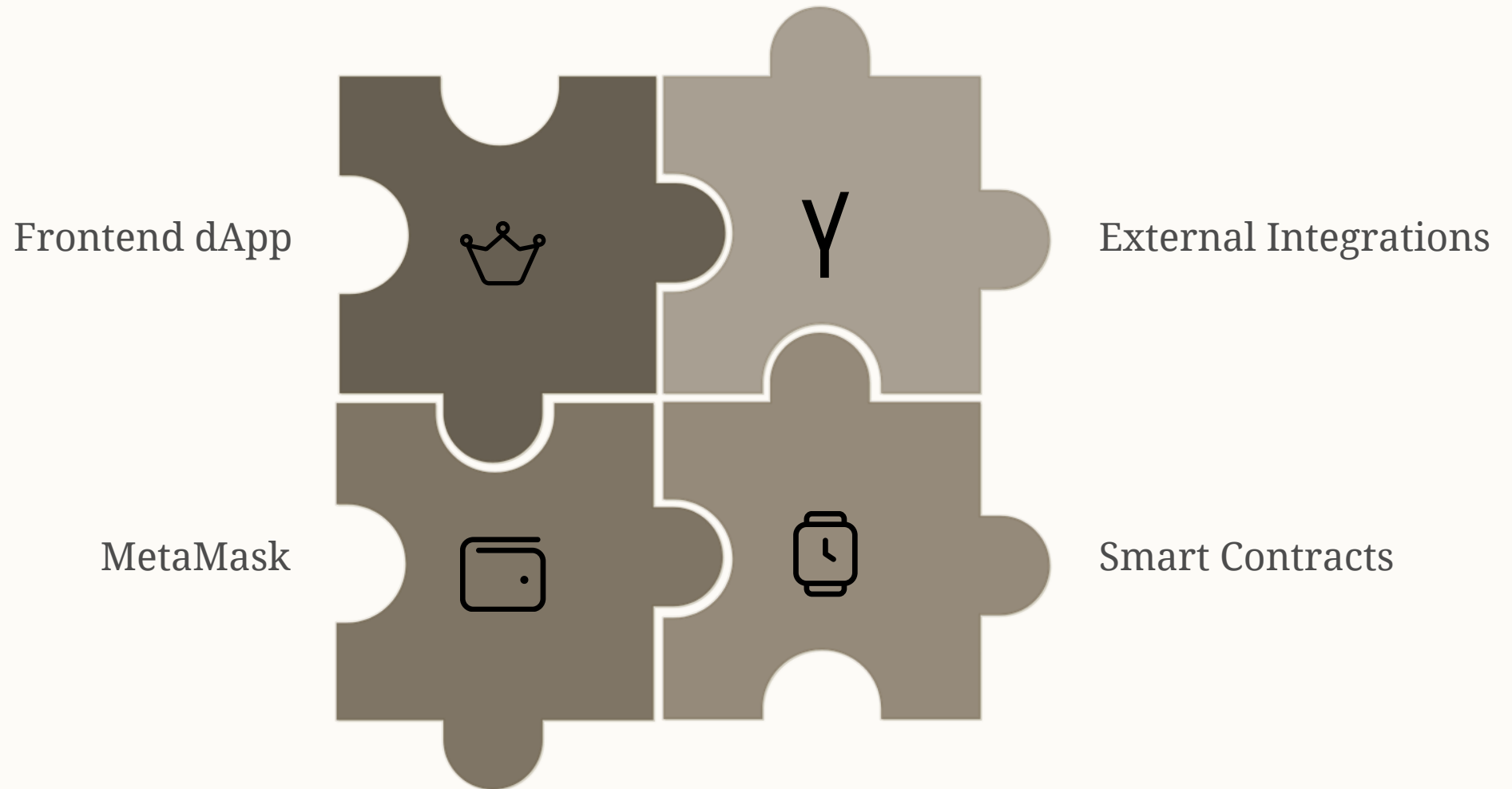


Diagram highlights: user flows through MetaMask to contracts; oracles feed price data; subgraph indexes events; deployment path supports Local Anvil demo then L2 testnets.

Smart Contracts — Modules

GovernanceToken

ERC20-like token with delegation, snapshot voting, and EIP-2612 permit support for gasless approvals.

AssetBadgeNFT

Minted per RWA with on-chain metadata pointers and attestations for provenance.

RwaVault

ERC4626-style deposit/withdraw and precise shares accounting with yield hooks.

ConstantProductAMM

$x*y=k$ pool with adjustable fee, slippage checks, and TWAP-compatible reserves.

RwaIssuerUpgradeable

UUPS upgradeable patterns with access control and initializer guards.

ProtocolFactory

Deterministic CREATE/CREATE2 deployment of vaults and AMMs for reproducible addresses.

DeFi & RWA Mechanics



Issue RWA Token

Deposit to Vault

Mint Vault Shares

Provide AMM Liquidity

Key mechanics: issuance maps off-chain asset to on-chain token; vault issues shares; AMM enables trading; oracle adapter enforces price freshness and bounds; treasury allocations visible in frontend for audits.

Governance Model

- Delegation: holders delegate voting power on-chain; snapshots used for proposal voting.
- Proposals: create with structured calldata, tracked by proposal ID.
- Voting: For / Against; votes tallied from contract state and exposed via subgraph.
- Timelock: Queue → delay → execute ensures review window and cancels if needed.

The screenshot displays a dark-themed web interface for a governance model, divided into two main panels: 'GOVERNANCE' and 'INDEXER'.

GOVERNANCE Panel:

- Proposal command center:** Features a dropdown menu for 'ID' set to '2'.
- Proposal List:** A table with three proposals, each with a status bar (green progress indicator):

ID	Proposal	Status
#1	Raise vault allocation cap	Active
#2	Update oracle signer set	Queued
#3	AMM fee to treasury	Review
- Selected Proposal #2:** A summary box for the 'Update oracle signer set' proposal, which is 'Active'. It shows 'FOR VOTES: 849 998' and 'AGAINST VOTES: 0', both labeled as 'Live counter'.
- Actions:** Two buttons at the bottom: a green 'Vote for' button and a red 'Vote against' button.

INDEXER Panel:

- Live activity stream:** A table showing recent system events:

Category	Event	Value	Time
VAULT	RWA deposit indexed	42,000 RWA	2m a
AMM	Pool rebalance simulated	1.8% slippage	18m a
GOV	Proposal queue ready	ETA 24h	41m a
ORACLE	NAV attestation refreshed	\$1.04	1h a

ⓘ Demo note: Local Anvil auto-creates test proposals; contract reverts with `ALREADY_VOTED` to prevent double voting.

Frontend Demo

Main dashboard

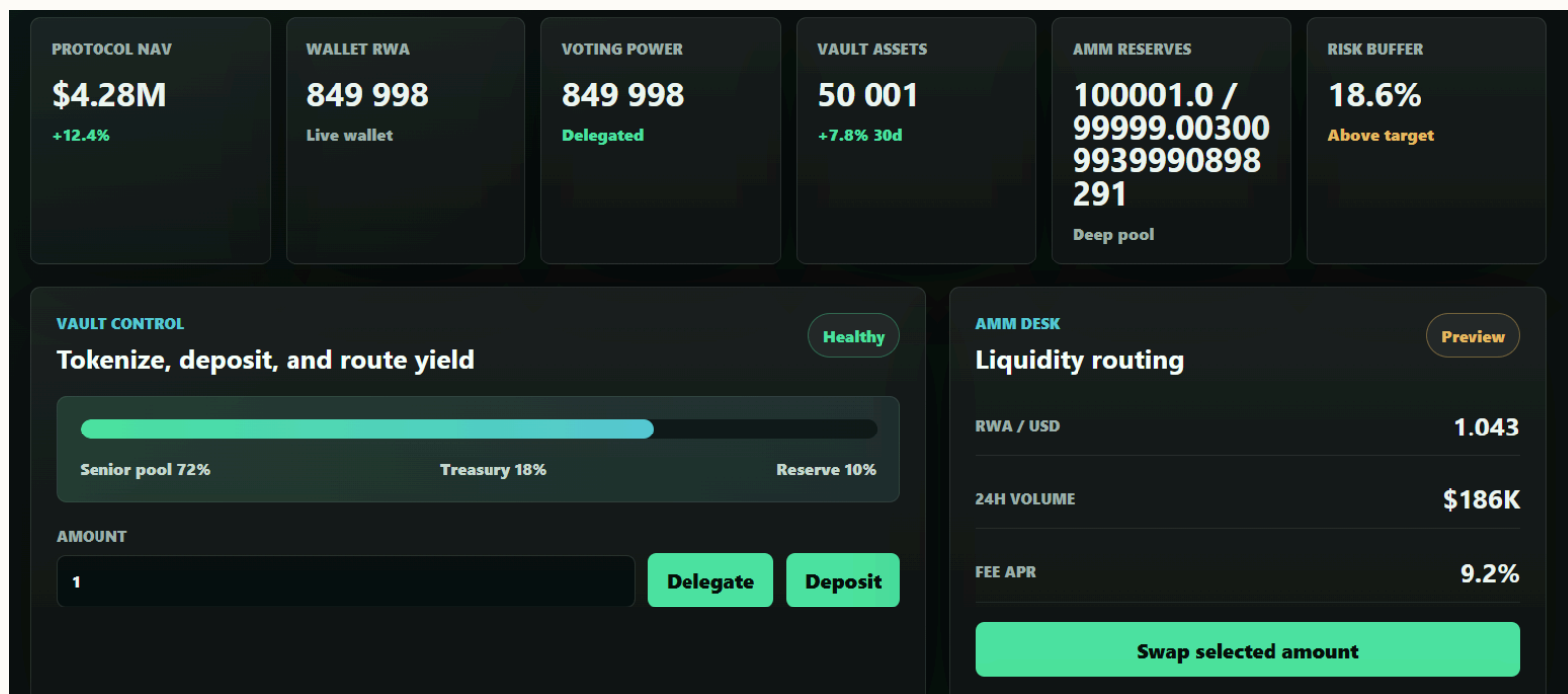
Shows wallet RWA balances, vault shares, AMM reserves, and treasury metrics.

Actions: Delegate · Deposit/Withdraw · Swap · Create Proposal · Vote

Governance panel

Livevote counts, proposal details, timelock queue, and execution controls (governor role gated).

Supports MetaMask, chain detection, and local Anvil environment switching.



Testing & Security Evidence



Foundry Tests

101 deterministic tests: unit, fuzz, and invariant suites. Fork tests target USDC, UniswapV2, Chainlink scenarios.



Coverage

~96% source coverage reported; gas reporting integrated for expensive paths.



Security scans & mitigations

Slither: 0 high/medium findings in main modules; patterns: reentrancy guards, checks-effects-interactions, access control, oracle stale checks, timelock enforced.

Deployment & Documentation

Deployment

npm run local:demo — automates Anvil spawn, deterministic deploys, and subgraph indexing on local node.

Frontend: <http://127.0.0.1:5173>

L2 path: prepared configs for Arbitrum / Base / Optimism (Sepolia), verifier scripts require funded deployer key.

Documentation

- README with quickstart and architecture overview
- Architecture docs: contract diagrams, flows, and upgrade strategy
- Audit report & gas report
- Coverage report, CI artifacts, and deployment runbook

Conclusion

- Full-stack application: Solidity contracts, subgraph indexing, Foundry tests.
- Multiple token standards and primitives: ERC20 governance token, ERC4626 vault, NFT attestations, AMM.
- Governance lifecycle: proposal creation → voting → timelock → execution with on-chain state visibility.
- Oracle & indexing integration: adapter enforces freshness; subgraph exposes audit trails.
- Testing & security: deterministic tests, high coverage, static analysis, and defense-in-depth patterns.
- Working local demo and clear L2 deployment path.

The project demonstrates a complete RWA protocol: tokenization, liquidity, governance, security, and frontend usability.
